

BÀI THỰC HÀNH

LAB 16

Điều khiển động cơ BLDC

Điều khiển BLDC thông qua ESC với EduFramework

S32K144 · MaaZEDU

Mục lục

1	Giới thiệu	2
1.1	Tổng quan bài lab	2
1.2	Mục tiêu	2
2	Kiến thức nền	2
2.1	BLDC và Electronic Speed Controller	2
2.2	Throttle command và arming	2
3	Thiết lập phần cứng	3
3.1	Phần cứng sử dụng	3
3.2	Ánh xạ chân điều khiển	3
3.3	Kết nối ESC và động cơ	3
4	EduFramework API	4
4.1	ESC_Init()	4
4.2	ESC_Arm()	5
4.3	ESC_SetThrottle()	5
5	Bài thực hành	5
5.1	Yêu cầu	5
5.2	Chương trình	5
5.3	Kiểm chứng	6
6	Mở rộng	7
6.1	Bài tập mở rộng	7
7	Tài liệu tham khảo	7

1 Giới thiệu

1.1 Tổng quan bài lab

Động cơ BLDC (Brushless DC Motor) sử dụng chuyển mạch điện tử thay cho chổi than và cổ góp cơ khí [1] [2]. Trong hệ thống của bài lab, bộ điều khiển tốc độ điện tử (Electronic Speed Controller - ESC) đảm nhiệm việc chuyển mạch và điều khiển công suất cho động cơ. S32K144 không điều khiển trực tiếp các pha của động cơ mà gửi tín hiệu điều khiển tới ESC.

Bài lab tập trung vào việc sử dụng API điều khiển ESC của EduFramework để khởi tạo, thực hiện quá trình khởi động an toàn (arming) và thay đổi mức ga (throttle), qua đó điều khiển hoạt động của động cơ BLDC.

1.2 Mục tiêu

MỤC TIÊU

Các mục tiêu của bài lab bao gồm:

1. Mô tả vai trò của BLDC và Electronic Speed Controller trong hệ thống điều khiển động cơ.
2. Giải thích ý nghĩa của throttle command và quá trình arming ESC.
3. Kết nối MaaZEDU, ESC, nguồn ngoài và động cơ BLDC đúng cấu hình thực hành.
4. Sử dụng `ESC_Init()`, `ESC_Arm()` và `ESC_SetThrottle()` để điều khiển BLDC.

2 Kiến thức nền

2.1 BLDC và Electronic Speed Controller

BLDC sử dụng rotor nam châm vĩnh cửu và các cuộn dây trên stator. Do không có chổi than cơ khí, dòng điện trong các cuộn dây cần được chuyển mạch bằng mạch điện tử để tạo mô-men quay [1] [2].

Trong bài lab, nhiệm vụ này được thực hiện bởi ESC. S32K144 không điều khiển trực tiếp ba đầu `U`, `V`, `W`; board chỉ gửi lệnh điều khiển tới ESC. ESC nhận nguồn công suất riêng qua `B+`, `B-` và điều khiển ba pha nối tới động cơ [3].

Ghi chú

Nguồn công suất của động cơ không được lấy trực tiếp từ GPIO của MaaZEDU. Nguồn ngoài phải phù hợp với ESC và động cơ đang sử dụng.

2.2 Throttle command và arming

EduFramework sử dụng lệnh throttle từ `0%` đến `100%`. Với cấu hình mặc định, `0%` tương ứng pulse width `1000 μs` và `100%` tương ứng `2000 μs`; các giá trị trung gian được ánh xạ tuyến tính trong khoảng này [6]. Throttle percentage là giá trị lệnh gửi tới ESC, không phải phần trăm tốc độ quay thực tế. RPM còn phụ thuộc vào động cơ, ESC, nguồn, tải và điều kiện vận hành. Nếu cần tốc độ thực tế, hệ thống phải sử dụng cơ chế đo như encoder.

Trước khi điều khiển motor, ESC cần được đưa về trạng thái khởi động an toàn. Trong EduFramework, `ESC_Arm()` gửi minimum throttle và giữ trạng thái đó trong `3000 ms` trước khi chương trình tiếp tục [6].

3 Thiết lập phần cứng

3.1 Phần cứng sử dụng

Bài lab sử dụng MaaZEDU Development Board với vi điều khiển S32K144 thuộc họ S32K1xx [4]. ESC được cấp nguồn riêng và điều khiển động cơ BLDC thông qua ba đầu ra pha.

Phần cứng	Mô tả
MaaZEDU Development Board	Board phát triển sử dụng vi điều khiển S32K144.
Electronic Speed Controller	Nhận RC PWM và điều khiển phần công suất của động cơ BLDC.
BLDC Motor	Động cơ ba dây pha được kết nối với các đầu U, V, W của ESC.
External DC Power Supply	Nguồn ngoài phù hợp với ESC và động cơ đang sử dụng.
Jumper wires	Kết nối GPIO2 và GND giữa MaaZEDU với ESC.
USB Cable	Kết nối board với máy tính để nạp chương trình.

Bảng 16.3.1. Phần cứng sử dụng trong bài thực hành

3.2 Ánh xạ chân điều khiển

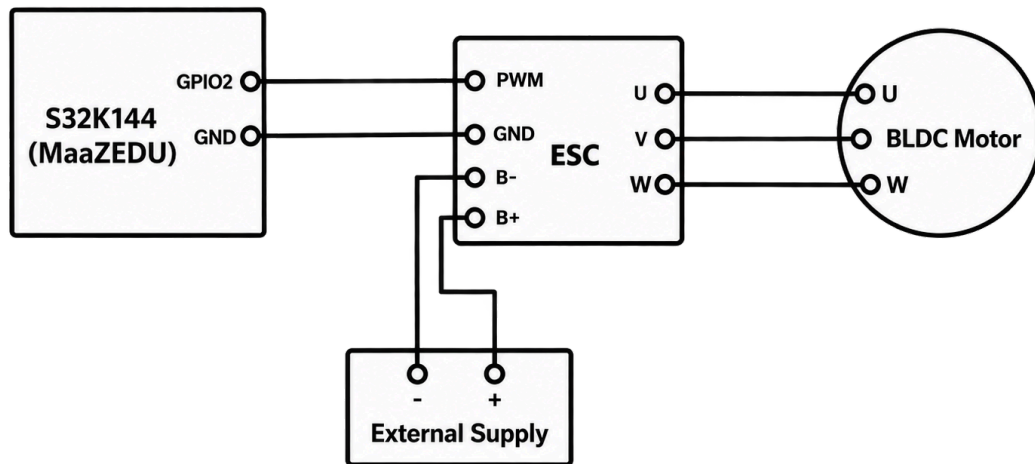
Bài thực hành sử dụng GPIO2 làm chân điều khiển ESC. Trên MaaZEDU, GPIO2 được ánh xạ tới PTD14 [7]. EduFramework xác nhận Logical Pin này hỗ trợ chức năng PWM cần thiết cho ESC [6].

Thành phần	Logical Pin	MCU Pin	Chức năng
ESC PWM input	GPIO2	PTD14	Tín hiệu điều khiển ESC

Bảng 16.3.2. Ánh xạ tín hiệu điều khiển ESC

3.3 Kết nối ESC và động cơ

GPIO2 được nối tới chân PWM của ESC và GND của MaaZEDU được nối tới GND phía điều khiển của ESC. Nguồn ngoài được nối tới B+ và B-. Ba đầu ra U, V, W của ESC được nối trực tiếp tới ba dây của động cơ BLDC.



Hình 16.3.1. Sơ đồ kết nối MaaZEDU, ESC và động cơ BLDC

Ghi chú

Ngắt nguồn công suất trước khi thay đổi dây nối. Cố định động cơ trước khi chạy và bắt đầu kiểm chứng ở mức throttle thấp. Không cấp nguồn nếu chưa xác nhận điện áp và khả năng cấp dòng phù hợp với ESC và động cơ.

4 EduFramework API

Bài thực hành chính sử dụng ba API của ESC Device: khởi tạo, arming và đặt throttle [6] .

4.1 ESC_Init()

ESC_Init

Khởi tạo ESC trên một Logical Pin hỗ trợ PWM.

Cú pháp

```
ESC_Init(pin);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
pin	Logical Pin	Chân điều khiển ESC, trong bài lab sử dụng GPIO2 .

Giá trị trả về

`true` nếu khởi tạo thành công; `false` nếu chân không phù hợp hoặc quá trình khởi tạo thất bại.

4.2 ESC_Arm()

ESC_Arm

Gửi minimum throttle và thực hiện khoảng arming của ESC.

Cú pháp

```
ESC_Arm();
```

Giá trị trả về

Không trả về giá trị.

Trong EduFramework, quá trình arming kéo dài `3000 ms` và hàm là blocking.

4.3 ESC_SetThrottle()

ESC_SetThrottle

Gửi throttle command tới ESC theo phần trăm.

Cú pháp

```
ESC_SetThrottle(percent);
```

Tham số

Tham số	Kiểu / giá trị chấp nhận	Mô tả
percent	Throttle	Giá trị từ <code>0%</code> tới <code>100%</code> ; giá trị lớn hơn <code>100%</code> được giới hạn về <code>100%</code> .

Giá trị trả về

Không trả về giá trị.

`ESC_SetThrottle()` điều khiển giá trị command gửi tới ESC; API này không đo RPM thực tế của động cơ.

5 Bài thực hành

5.1 Yêu cầu

Xây dựng chương trình điều khiển động cơ BLDC thông qua ESC nối với `GPIO2`. Sau khi khởi tạo và arming, chương trình thay đổi throttle theo chuỗi:

`20% → 40% → 60% → 40% → 20% → 0%`

Mỗi mức throttle được giữ trong `3000 ms`. Sau khi trở về `0%`, chuỗi được lặp lại liên tục. Mục tiêu là quan sát phản ứng của động cơ khi throttle command tăng và giảm, không yêu cầu đo RPM trong bài thực hành này.

5.2 Chương trình

Trong `src/main.c`, triển khai chương trình đã được kiểm chứng trên phần cứng như sau:

```
#include "Arduino.h"
#include "esc.h"

int main(void)
{
    setup();

    ESC_Init(GPIO2);
    ESC_Arm();

    while (1)
    {
        ESC_SetThrottle(20U);
        delay(3000U);

        ESC_SetThrottle(40U);
        delay(3000U);

        ESC_SetThrottle(60U);
        delay(3000U);

        ESC_SetThrottle(40U);
        delay(3000U);

        ESC_SetThrottle(20U);
        delay(3000U);

        ESC_SetThrottle(0U);
        delay(3000U);
    }

    return 0;
}
```

Mã nguồn 16.5.1. Chương trình điều khiển BLDC theo chuỗi throttle

`setup()` khởi tạo EduFramework. `ESC_Init(GPIO2)` chuẩn bị đường điều khiển, `ESC_Arm()` thực hiện arming, sau đó `ESC_SetThrottle()` lần lượt gửi các mức throttle đã quy định. `delay(3000U)` giữ mỗi mức trong ba giây để dễ quan sát [5].

Bài thực hành không sử dụng encoder feedback nên chỉ kiểm chứng phản ứng của motor theo command, không xác định RPM thực tế.

5.3 Kiểm chứng

Build và nạp chương trình xuống MaaZEDU. Kiểm tra `GPIO2`, `GND`, `B+`, `B-` và ba dây `U/V/W` trước khi cấp nguồn công suất. Sau arming, quan sát motor khi throttle tăng từ 20% tới 60% rồi giảm về 0%.

KẾT QUẢ MONG ĐỢI

ESC hoàn thành quá trình arming trước khi motor nhận chuỗi throttle. Động cơ phản ứng theo các mức command 20%, 40%, 60%, sau đó giảm về 40%, 20% và dừng ở 0%. Chuỗi được lặp lại liên tục. Kết quả cần thể hiện sự thay đổi ổn định theo command, không yêu cầu quan hệ tuyến tính giữa throttle percentage và RPM thực tế.

6 Mở rộng

Bên cạnh ba API sử dụng trong bài chính, EduFramework cung cấp thêm các API để cấu hình và theo dõi command của ESC [6] .

API	Mục đích
<code>ESC_SetPulseRange()</code>	Thay đổi minimum và maximum pulse dùng để ánh xạ 0% đến 100% throttle.
<code>ESC_SetMicroseconds()</code>	Gửi trực tiếp pulse width theo microsecond trong khoảng đã cấu hình.
<code>ESC_GetThrottle()</code>	Đọc lại throttle command gần nhất được lưu trong phần mềm.
<code>ESC_GetMicroseconds()</code>	Đọc lại pulse width command gần nhất.
<code>ESC_IsInitialized()</code>	Kiểm tra ESC Device đã được khởi tạo hay chưa.
<code>ESC_End()</code>	Dừng output do ESC Device quản lý và reset trạng thái nội bộ.

Bảng 16.6.1. Các ESC API nâng cao

Ghi chú

Các hàm `ESC_GetThrottle()` và `ESC_GetMicroseconds()` chỉ trả về command đã gửi, không phải tốc độ hoặc phản hồi vật lý từ động cơ. `ESC_SetPulseRange()` chỉ nên thay đổi khi khoảng điều khiển của ESC thực tế đã được xác định.

6.1 Bài tập mở rộng

Thay chuỗi throttle cố định bằng potentiometer nối tới Analog Input. Đọc giá trị ADC, quy đổi thành throttle command trong một giới hạn phù hợp, sau đó sử dụng `ESC_SetThrottle()` để điều khiển động cơ.

Potentiometer → `analogRead()` → scale ADC → `ESC_SetThrottle()` → BLDC

7 Tài liệu tham khảo

- [1] James L. Kirtley Jr., *Course Notes 7: Permanent Magnet “Brushless DC” Motors*, MIT OpenCourseWare - 6.685 Electric Machines, 2013.
- [2] Deepak Mohanraj et al., *A Review of BLDC Motor: State of Art, Advanced Control Techniques, and Applications*, IEEE Access, Vol. 10, pp. 54833-54869, 2022.
- [3] Luc Hanneuse, *Output Devices - Brushless Motor and ESC*, Fab Academy - Sorbonne Lab, 2019.
- [4] NXP Semiconductors, *S32K1xx MCU Family - Data Sheet*, S32K1XX, Rev. 15, 2026.
- [5] Arduino, *Arduino Language Reference*, Arduino Documentation.
- [6] EduFramework, *ESC Device API*, EduFramework Source Code.
- [7] MaaZEDU, *MaaZEDU Development Board Guide*.